Model Checking of Knowing How Logics

Rens Doodeman, Ziqi Wang, and Zixuan Chen

Wednesday 18th March, 2026

Abstract

TODO!

Contents

1 How to use this? **2**

2 Introduction **2**

3 Single Agent **3**

4 Multi Agent **6**

5 Tests for Semantics **12**

6 Conclusion **13**

Bibliography **13**

1 How to use this?

To generate the PDF, open `report.tex` in your favorite LaTeXeditor and compile. Alternatively, you can manually do `pdflatex report; bibtex report; pdflatex report; pdflatex report` in a terminal.

You should have stack installed (see `https://haskellstack.org/`) and open a terminal in the same folder.

- To compile everything: `stack build`.
- To open ghci and play with your code: `stack ghci`
- To run the executable from Section **??**: `stack build && stack exec myprogram`
- To run the tests from Section **??**: `stack clean && stack test --coverage`

2 Introduction

Ability-based epistemic logics are found many places in computer science, as arguably they provide us with a formal approach for strategic reasoning and AI planning. Several logics in this family have been studied lately. In particular, epistemic logics describing the notion of *knowing how* received a lot of attention, due to their simplicity and good meta-logical properties.

The simplest version of *knowing how*, $\mathcal{L}_{Kh}$, based on the paper by (Wang 2015[3]), with the semantics reformulated in (Areces et al. 2021[1]). This logic captures the agent's ability to guarantee the completion of a task under a given condition. This idea is ideal in a sense, since even though the agent manages to do something under a condition, it does not imply the agent really know how. For example, one can win a lottery does not imply one can guarantee how to win. The uncertainty is involved in this case. Therefore, another logic may be prefered, $\mathcal{L}^U_{Kh}$. In this logic, an agent may have *indistinguishability* among several plans they might take, a critical concept in epistemic logic. Indistinguishable plans form equivalence classes, partitioning the set of available plans. Intuitively, plans considered identical by the agent are placed in the same class. Furthermore, this logic involves multi-agents, as opposed to a single agent. Finally, a new logic, $reg - \mathcal{L}^U_{Kh}$, a variation of the second version, introduces equivalence classes that are defined in terms of finite automatons.

In this project, we aim to model this family of knowing-how logics. We also intend to implement the model checking for the logic $reg - \mathcal{L}^U_{Kh}$ defined in [2].

Our project goals are defined as follows.

- Get familiar with the family of knowing-how logics;
- Implement the the family of knowing-how logics (syntax, semantics) in Haskell;
- Implement the explicit model checker for the basic knowing how logic;
- Implement model checking problem for the logic $reg - \mathcal{L}^U_{Kh}$ defined in [2];
- (Optional) Implement testing for model checking.
- (Optional) Implement a user-interface or at the very least an executable (using parsing) for model checking.

3 Single Agent

In this section we model the language of Knowing How L_{KH} as by the definition of Y. Wang [3].

Given a set of proposition letters P, the language L_{KH} is defined as follows s.t. $Kh(\psi, \varphi)$ is the modality expressing "the agent knows how to achieve φ given ψ". The abbreviations $\bot$, $\vee$, $\rightarrow$, and the universal modality $U_\varphi := Kh(\neg\varphi, \bot)$ have been left out for simplicity.

```
module SingleAgent where

import Data.List (nub, delete)
-- import NoneEmpty including its constructor :|
import Data.List.NonEmpty (NonEmpty(..), toList)
import Test.QuickCheck

type Proposition = Int

data Form = P Proposition | Neg Form | Conj Form Form | KH Form Form | T
    deriving (Eq, Show, Ord)
```

Instead of a Kripke semantics, the Logic opts for a labelled transition system (LTS) $(\mathcal{S}, \mathcal{R}, \mathcal{V})$ s.t.

- $\mathcal{S}$ is a non-empty set of states
- $\mathcal{R} : \Sigma \rightarrow 2^{\mathcal{S}\times\mathcal{S}}$ is a collection of transitions labelled by actions in Σ
- $\mathcal{V} : \mathcal{S} \rightarrow 2^P$ is a valuation function

Additionally $s \xrightarrow{a} t$ if $(s, t) \in R(a)$, being the relation or 'transition' from s to t. This is further modelled after this code snippet. In the literature by Y. Wang the LTS is called an ability map.

When creating a LTS, consider that states :: NonEmpty State, and therefore must use the :| constructor, i.e. (n :| [n+1..]). For more information see NonEmpty on Hoogle.

```
type Action = Integer
type Plan = [Action]
type State = Int
type Valuation = [(State, [Proposition])]

data AbilityMap = LTS {
    states :: NonEmpty State,
    transitions :: Relations,
    valuation :: Valuation
} deriving(Show)
```

In (Wang2015), two relational notions are introduced:

- R_a, the atomic relation associated with an action a;
- R_σ, the composite relation associated with a plan σ.

Both of these are binary relations on states. Therefore, in our implementation, we use the same Haskell type `Rel` to represent them, since both can be understood as sets of pairs of states.

To represent the family of atomic relations $R = (R_a)_a$, we index relations by actions. This gives rise to the type `Relations`, corresponding to the collection of action-labelled transitions in the literature.

We do not model R_σ explicitly as a separate data structure. Instead, we only implement the operations needed to reason about plan execution and strong executability.

For this purpose, we define the following helper functions:

- `image`, which computes the image of a state under a relation, i.e. it collects all successor states of a given state;
- `r_a`, which takes an action a and the family of relations R, and returns the indexed relation R_a;
- `executePlan`, which computes all possible end states obtained after executing a plan from a given initial state;
- `stronglyExecutableAt`, which determines whether a plan is strongly executable at a given state.

```
-- Both $R_a$ and $R_\sigma$ share the same type.
-- Here we model the whole set of relations, without classification.
type Rel = [(State, State)]

-- The family of action-indexed relations (R_a)_a
type Relations = [(Action, Rel)]

-- Image of a state under R_a
image :: Rel -> State -> [State]
image r u = [v | (x, v) <- r, x == u]

-- Given (R_a)_a and an action x, return R_x
r_a :: Relations -> Action -> Rel
r_a rs a =
    case lookup a rs of
        Just r  -> r
        Nothing -> []

-- Execute a plan from a given start state
-- This corresponds to R_sigma(u)
executePlan :: Relations -> State -> Plan -> [State]
executePlan _  u []       = [u]
executePlan rs u (a:sigma) =
    nub (concat [ executePlan rs v sigma | v <- image (r_a rs a) u ])

-- Plan is SE or not at a state
stronglyExecutableAt :: Relations -> State -> Plan -> Bool
stronglyExecutableAt _  _ []       = True
stronglyExecutableAt rs u (a:sigma) =
    let next = image (r_a rs a) u
    in  not (null next) &&
        all (\v -> stronglyExecutableAt rs v sigma) next
```

The satisfaction relation $\models$ defines when a formula φ is true in a given state s of an ability map $\mathcal{M}$. Before specifying the semantics, we first introduce two helper functions for the semantics of the Kh operator.

```
-- Given an LST and formula, returns the states that satify said formula
statesSatisifying :: AbilityMap -> Form -> [State]
statesSatisifying m f = [s | s <- toList (states m), isTrue (m, s) f]

-- Given an LST, find all plans.
findPlans :: AbilityMap -> [Plan]
findPlans m = nub (concatMap (plansFrom depth) (toList (states m)))
  where
    -- For now we limit the depth to avoid infinite loops.
    depth = 5

    -- Flatten transitions
    edges :: [(Action,(State,State))]
    edges = [ (a,(u,v))| (a,rel) <- transitions m, (u,v) <- rel ]
```

```haskell
    -- Generate all plans from a given state up to our specified depth
    plansFrom :: Int -> State -> [Plan]
    plansFrom 0 _ = []
    plansFrom d s =
        [ [a] | (a,(x,_)) <- edges, x == s ] -- single action plans
        ++ [ a:p | (a,(x,y)) <- edges, x == s, p <- plansFrom (d-1) y ]

isTrue :: (AbilityMap, State) -> Form -> Bool
isTrue _ T = True
isTrue (m, s) (P p) =
  case lookup s (valuation m) of
    Just props -> p `elem` props
    Nothing -> False
isTrue (m, s) (Neg f) = not (isTrue (m, s) f)
isTrue (m, s) (Conj f g) = isTrue (m, s) f && isTrue (m, s) g
-- KH is NOT local; its truth does not depend on the state at which it is evaluated.
-- KH either holds at all states, or none of them.
isTrue (m, _) (KH f g) =
    any (\a ->
        all (\s ->
            stronglyExecutableAt rs s a
            && all (\t -> isTrue (m,t) g) (executePlan rs s a)
        ) statesF
    ) candidatePlans
  where
    rs = transitions m
    statesF = statesSatisifying m f
    candidatePlans = findPlans m

-- Infix alias for the satisfaction relation
(|=) :: (AbilityMap, State) -> Form -> Bool
(|=) = isTrue
```

Finally, for this section we define the instances of Arbitrary for `Form` and `Arbitrary` respectively. For `Form` we use the `sized` function to ensure that the generated formulas remain finite in size.

```haskell
instance Arbitrary Form where
    arbitrary = sized randomForm where
        -- Helper function to generate random formulas of a given size
        randomForm :: Int -> Gen Form
        randomForm 0 = oneof [P <$> choose (1, 5), return T]
        randomForm n = oneof
            [ P <$> choose (1, 5)
            , return T
            , Neg <$> randomForm (n - 1)
            , Conj <$> randomForm (n `div` 2) <*> randomForm (n `div` 2)
            , KH <$> randomForm (n `div` 2) <*> randomForm (n `div` 2)
            ]

instance Arbitrary AbilityMap where
    arbitrary = do
        n <- choose (1,10)
        let sts = [1..n] -- n states
        rels <- generateRelations n sts
        vals <- mapM generateValuation sts
        return $ LTS (head sts :| tail sts) rels vals
        where
            generateRelations n sts
                | n == 1 = return []
                | otherwise = do
                    m <- choose (1,n) -- decide how many actions to generate
                    actions <- vectorOf m (choose (1,5))
                    mapM (generateRel sts) actions

             -- for each action a, generate a relation labeled by a
            generateRel sts a = do
                x <- elements sts
                y <- elements (delete x sts) -- avoid loops
                return (a, [(x,y)])

            -- for each state s, generate a list of propositions
            generateValuation s = do
```

```
            props <- listOf (choose (1,5))
            return (s, nub props)
```

For now, we haven't had time yet to implement any 'interactive' elements. But it is possible to try out semantics using the definitions by running `stack ghci`. For example:

```
ghci> import Test.QuickCheck
ghci> import Data.List.NonEmpty (NonEmpty(..), toList)

ghci> generate (arbitrary:: Gen AbilityMap)
LTS {
    states = 1 :| [2,3,4,5,6],
    transitions = [(3,[(5,2)]),(4,[(2,4)]),(1,[(5,4)])],
    valuation = [(1,[4,2]),(2,[]),(3,[1]),(4,[3]),(5,[4]),(6,[])]
    }

ghci> m = ...

ghci> isTrue (m, 2::State) (KH (Conj (P 4) (Neg (P 2))) (P 3))
True

ghci> isTrue (m, 2::State) (KH (P 4) (P 1))
False
```

- HLINT ignore "Eta reduce" -

4 Multi Agent

In this section we model the language of Uncertainty-based Knowing How with regularity constraints $reg - \mathcal{L}^U_{KH}$ [2]. The paper we reference combines and alters already existing languages into the one we model here. The first key difference from the previous language is that it moves from a single-agent language to a multi-agent one.

Given a set of proposition letters $Prop$ and a set of agents Agt, where $p \in Prop$ and $i \in Agt$, the language $reg - \mathcal{L}^U_{KH}$ is defined as

$$\varphi := p \mid \neg\varphi \mid \varphi \vee \varphi \mid Kh_i(\psi, \varphi)$$

s.t. $Kh_i(\psi, \varphi)$ can be interpreted as "when ψ is the case, the agent i knows how to make φ true".

The semantics of this language is defined as follows. Let $\mathcal{S} = (S, (R_a)_{a \in Act}, (U_i)_{i \in Agt}, V)$ be a finite reg-LTS^U, and $s \in S$. The satisfaction relation $\models$ is defined as:

$\mathcal{S}, s \models p$	*iff*	$p \in V(s)$
$\mathcal{S}, s \models \neg\varphi$	*iff*	$\mathcal{S}, s \not\models \varphi$
$\mathcal{S}, w \models \psi \vee \varphi$	*iff*	$\mathcal{S}, w \models \psi$ or $\mathcal{S}, w \models \varphi$
$\mathcal{S}, s \models Kh_i(\psi, \varphi)$	*iff*	there is a finite automaton $\mathcal{A} \in U_i$, such that (1) $[\![\psi]\!]^{\mathcal{S}} \subseteq SE(L(\mathcal{A}))$ and (2) for every $t \in [\![\psi]\!]^{\mathcal{S}}$, $R_{L(\mathcal{A})}(t) \subseteq [\![\varphi]\!]^{\mathcal{S}}$,

where $[\![\psi]\!]^{\mathcal{S}} := \{w \in S \mid \mathcal{S}, w \models \psi\}$ is the set of all states satisfying ψ.

```
module MultiAgent where
import SingleAgent
  ( Proposition
  , Action
  , Plan
  , State
  , Valuation
  , Relations
  , executePlan
  , stronglyExecutableAt
  , image
  , r_a
  )

import Data.List (nub)
import Data.Maybe (fromMaybe)

type Agent = Int

data RegForm = P Proposition | Neg RegForm | Disj RegForm RegForm | KH Agent RegForm RegForm
    deriving (Eq, Show, Ord)
```

Now we define the automaton used in the model-checking procedure. Formally, a non-deterministic finite automaton is a tuple $\mathcal{A} = (Q, Act, \delta, I, F)$, where:

- Q is a finite set of states,
- Act is a finite alphabet of actions,
- $\delta \subseteq Q \times Act \times Q$ is the transition relation,
- $I \subseteq Q$ is the set of initial states,
- $F \subseteq Q$ is the set of accepting (final) states.

Since the automaton considered here is non-deterministic, its transition component is modeled as a relation rather than as a function. For the special automaton $\mathcal{A}_{(t_1,t_2)}$ used later in Condition (2), we define its transitions by mirroring the LTS relations: for all $t, t' \in Q$ and $a \in Act$,

$$t \xrightarrow{a} t' \in \delta \iff (t, t') \in R_a.$$

```
type Successors = [State]
type ActionAtState = (State, Action)

data Automaton = ATMN {
    statesA :: [State],
    actionsA :: [Action],
    transitionsA :: [(ActionAtState, Successors)],
    initial :: [State],
    final :: [State]
} deriving (Eq, Show, Ord)
```

For $\pi \subseteq Act^*$, $u \in W$ and $U \subseteq W$, define:

$$R_\pi := \bigcup_{\sigma \in \pi} R_\sigma, \qquad R_\pi(u) := \bigcup_{\sigma \in \pi} R_\sigma(u), \qquad R_\pi(U) := \bigcup_{u \in U} R_\pi(u)$$

A set of plans $\pi \subseteq Act^*$ is *strongly executable* at $u \in W$ iff every plan $\sigma \in \pi$ is strongly executable at u. This gives rise to the definition $SE(\pi) := \bigcap_{\sigma \in \pi} SE(\sigma)$, collecting all states where every plan in π is strongly executable.

```
type PlanSet = [Plan]

-- R_pi(u) = union of R_sigma(u) for all sigma in pi
rPiU :: Relations -> State -> PlanSet -> [State]
rPiU rs u plans =
    nub (concat [executePlan rs u sigma | sigma <- plans])

-- R_pi(X) = union of R_pi(u) for all u in X
rPiX :: Relations -> [State] -> PlanSet -> [State]
rPiX rs xs plans =
    nub (concat [rPiU rs u plans | u <- xs])

-- SE(sigma) = set of all states at which sigma is strongly executable
seSigma :: [State] -> Relations -> Plan -> [State]
seSigma sts rs sigma =
    [u | u <- sts, stronglyExecutableAt rs u sigma]

-- SE(pi) = intersection of SE(sigma) for all sigma in pi
sePi :: [State] -> Relations -> PlanSet -> [State]
sePi sts rs plans =
    [u | u <- sts, all (stronglyExecutableAt rs u) plans]
```

In terms of automata, $\mathrm{SE}(L(\mathcal{A})) := \bigcap_{\sigma \in L(\mathcal{A})} \mathrm{SE}(\sigma)$ collects all states at which every plan in $L(\mathcal{A})$ is strongly executable, and $R_{L(\mathcal{A})} := \bigcup_{\sigma \in L(\mathcal{A})} R_\sigma$ collects all transitions realisable by some plan in $L(\mathcal{A})$. The functions above are not used directly in the model-checking algorithm, but are included to align with the notions from the paper.

Each agent has sets of indistinguishable plans, forming equivalence classes. Since plans are finite sequences of actions, they can be viewed as strings over Act. From this perspective, each equivalence class can be represented as the language of a finite automaton $\mathcal{A}$, and a witness for Kh_i is a string in $L(\mathcal{A})$.

A reg-LTS^U is a tuple $\mathcal{S} = (S, (R_a)_{a \in Act}, (U_i)_{i \in Agt}, V)$, where the elements of U_i associated with an agent i are *finite automata*, namely, $\emptyset \neq U_i = \{\mathcal{A}_1, \mathcal{A}_2, \ldots\}$. Here, $L(\mathcal{A}_i)$ denotes the language accepted by $\mathcal{A}_i$. For every $\mathcal{A}_m, \mathcal{A}_n \in U_i$ such that $m \neq n$, we have $L(\mathcal{A}_m) \cap L(\mathcal{A}_n) = \emptyset$.

```
type Uncertainty = [(Agent, [Automaton])] -- U_i = {A_1,...}, for each agent i

data RegLTSU = RegLTSU{
        statesM :: [State],
        relationsM :: Relations,
        uncertainty :: Uncertainty,
        valuationM :: Valuation -- use the Valuation from singleAgent
    } deriving (Eq, Show, Ord)

-- Given an agent i, return U_i = [A_1,...]
getAgentAuts :: RegLTSU -> Agent -> [Automaton]
getAgentAuts m agent =
    fromMaybe [] (lookup agent (uncertainty m))

-- TODO: Shall we write a checker to check A_i's are indeed automata. Namely, are the language of them really
     pairwise disjoint? Or simply leave it as an assumption?
```

Now we define the product digraph G used in the model-checking procedure. Given a reg-LTS^U $\mathcal{S}$ and an automaton $\mathcal{A} = (Q, Act, \delta, I, F)$, we construct a digraph $G = (V, E)$ as follows:

- $V = Q \times S$, representing the combined states of the automaton and the underlying transition system;
- E is the set of edges such that $((q, t), (q', t')) \in E$ if and only if there exists an action $a \in Act$ such that:

1. $q \xrightarrow{a} q'$ in $\mathcal{A}$;
2. $(t, t') \in R_a$.

This construction synchronizes transitions of the automaton with transitions in the LTS under the same action, allowing us to track which runs of the automaton are realizable in the model.

```
type GVertex = (State, State) -- (Automaton State, LTS State)
type GEdge = (GVertex, GVertex)

data Digraph = Digraph {
    vSet :: [GVertex],
    eSet :: [GEdge]
} deriving (Eq, Show, Ord)

-- Helper function to get next automaton states under a given action
getAutNext :: Automaton -> State -> Action -> [State]
getAutNext atmn q a =
    fromMaybe [] (lookup (q, a) (transitionsA atmn))

-- Helper function to get next LTS states under a given action
getLtsNext :: RegLTSU -> State -> Action -> [State]
getLtsNext m t a = image (r_a (relationsM m) a) t

-- Build the product digraph G = (V, E)
buildDigraph :: RegLTSU -> Automaton -> Digraph
buildDigraph m atmn = Digraph nodes edges
  where
    nodes = [(q, s) | q <- statesA atmn, s <- statesM m]
    edges = [ ((q, t), (q', t'))
            | (q, t) <- nodes
            , act    <- actionsA atmn
            , q'     <- getAutNext atmn q act
            , t'     <- getLtsNext m t act
            ]
```

Given a finite reg-LTS^U $\mathcal{S} = (S, (R_a)_{a \in Act}, (U_i)_{i \in Agt}, V)$, a state $s \in S$, and a formula $Kh_a(\varphi, \psi)$, we check whether $\mathcal{S}, s \models Kh_a(\varphi, \psi)$ by iterating over all automata $\mathcal{A} \in U_a$ and verifying two conditions:

1. $[\![\varphi]\!]^{\mathcal{S}} \subseteq \mathrm{SE}(L(\mathcal{A}))$
2. $R_{L(\mathcal{A})}([\![\varphi]\!]^{\mathcal{S}}) \subseteq [\![\psi]\!]^{\mathcal{S}}$

where $[\![\varphi]\!]^{\mathcal{S}}$ is the set of states in $\mathcal{S}$ at which φ holds. Since $L(\mathcal{A})$ may be infinite, directly enumerating all plans is not feasible. Following [2], we handle each condition via a separate algorithm, both of which reduce the problem to reachability checks on finite graphs.

To perform the reachability checks in PTIME, we use the following *depth-first search* algorithm:

Require: Digraph $G = (V, E)$, start vertex v_0, target set $T_0 \subseteq V$

$v \leftarrow v_0$
$T \leftarrow T_0$
$visited \leftarrow \emptyset$
function DFS($G, v, T, visited$)
    if $v \in T$ **then return true**
    end if
    if $v \in visited$ **then return false**
    end if
    $visited \leftarrow visited \cup \{v\}$

```
    for w ∈ successors(G, v) do
        if DFS(G, w, T, visited) then return true
        end if
    end for
    return false
end function
```

In Haskell:

```haskell
-- Successors of a vertex in the digraph
successors :: Digraph -> GVertex -> [GVertex]
successors g v = [v' | (u, v') <- eSet g, u == v]

-- DFS reachability: is any vertex in targetSet reachable from start?
-- The type can be read as: digraph -> start -> targets -> visited_set -> Bool
dfsReachable :: Digraph -> GVertex -> [GVertex] -> [GVertex] -> Bool
dfsReachable _ start targetSet _
    | start `elem` targetSet = True
dfsReachable _ start _ visited
    | start `elem` visited = False
dfsReachable g start targetSet visited =
    any (\v -> dfsReachable g v targetSet (start : visited)) (successors g start)
```

The algorithm that checks Condition (1) from [2] works as follows:

Require: reg-LTS^U $\mathcal{S}$, state $s \in S$, automaton $\mathcal{A} = (Q, Act, \delta, I, F)$

function CHECKSE-ORIGINAL($\mathcal{S}, s, \mathcal{A}$)
  Build the product digraph $G = (Q \times S,\ E)$
  for $a \in Act$ **do**
    for $t \in S$ such that $R_a(t) = \emptyset$ **do**
      for $q \in Q$ such that $\delta(q, a) \neq \emptyset$ **do**
        for $q_0 \in I$ **do**
          if (q, t) is reachable from (q_0, s) in G **then**
            return true
          end if
        end for
      end for
    end for
  end for
  return false
end function

Instead of using the nested for-loops, we first collect all *bad vertices* via set comprehension, i.e.

$$Bad = \{(q, t) \in Q \times S \mid \exists\, a \in Act : \delta(q, a) \neq \emptyset \text{ and } R_a(t) = \emptyset\},$$

and then perform a single DFS reachability check from each initial vertex (q_0, s) to the set Bad. This is equivalent to the original procedure, but avoids repeated reachability checks.

A bad vertex (q, t) represents a situation where the automaton expects to perform some action a at state q, but the LTS cannot execute a at state t. If such a vertex is reachable, it means that some plan in $L(\mathcal{A})$ fails to be strongly executable.

Remark: This function might be more accurately called `checkNotSE` *rather than* `checkSE`*, since it returns true exactly when strong executability fails.*

Require: reg-LTS^U $\mathcal{S} = (S, (R_a), (U_i), V)$, state $s \in S$, automaton $\mathcal{A} = (Q, Act, \delta, I, F)$

function CHECKSE($\mathcal{S}, s, \mathcal{A}$)

Build product digraph $G = (Q \times S,\ E)$
$Bad \leftarrow \{(q, t) \in V \mid \exists\, a \in Act : \delta(q, a) \neq \emptyset \text{ and } R_a(t) = \emptyset\}$
for $q_0 \in I$ **do**
  if DFS((q_0, s)) with target Bad **then return true**
  end if
end for
return false
end function

Require: reg-LTS^U $\mathcal{S}$, automaton $\mathcal{A}$, truth set $[\![\varphi]\!]^{\mathcal{S}}$
function CheckCond1($\mathcal{S}, \mathcal{A}, [\![\varphi]\!]^{\mathcal{S}}$)
for $s \in [\![\varphi]\!]^{\mathcal{S}}$ **do**
  if CheckSE($\mathcal{S}, s, \mathcal{A}$) **then return false**
  end if
end for
return true
end function

In Haskell:

```haskell
-- Compute the set of bad vertices in G.
-- A vertex (q, t) is bad iff there exists an action a such that:
--   (i)  the automaton has a transition from q under a, and
--   (ii) the LTS has no successor from t under a.
badVertices :: RegLTSU -> Automaton -> [GVertex]
badVertices m atmn =
    nub [ (q, t)
        | q <- statesA atmn,
        t <- statesM m,
        a <- actionsA atmn,
        not (null (getAutNext atmn q a)),  --  delta(q,a) != empty
        null (getLtsNext m t a)            --  R_a(t) == empty
        ]

-- checker whether there is a path from (q0, s) to a bad vertex.
-- If so, return True. This means that s not in SE(L(A)).
-- Else, return False. This means that s in SE(L(A)).
checkSE :: RegLTSU -> State -> Automaton -> Bool
checkSE m s atmn =
    let g   = buildDigraph m atmn
        bad = badVertices m atmn
    in  any (\q0 -> dfsReachable g (q0, s) bad []) (initial atmn)

checkCond1 :: RegLTSU -> Automaton -> [State] -> Bool
checkCond1 m atmn =
    all (\s -> not (checkSE m s atmn))
```

```haskell
-- Construct A_(t1, t2)
-- This automaton accepts all plans from t1 to t2
buildPathAutomaton :: RegLTSU -> State -> State -> Automaton
buildPathAutomaton m t1 t2 = ATMN
    { statesA      = statesM m,
    actionsA     = acts,
    transitionsA = [ ((s, a), getLtsNext m s a) | s <- statesM m, a <- acts],
    initial      = [t1],
    final        = [t2]
    }
  where
    -- Collect all actions that appear in the LTS relations
    acts = nub [a | (a, _) <- relationsM m]

-- Check the intersection of L(A1) and L(A2) is empty or not
intersectionNonEmpty :: Automaton -> Automaton -> Bool
intersectionNonEmpty a1 a2 =
    any (\sv -> dfsReachable prodGraph sv targets []) startVs
  where
```

```
    -- Shared actions
    sharedActs = nub [a | a <- actionsA a1, a `elem` actionsA a2]

    -- Product graph vertices and edges
    prodNodes = [(q1, q2) | q1 <- statesA a1, q2 <- statesA a2]
    prodEdges = [ ((q1, q2), (q1', q2'))
                | (q1, q2) <- prodNodes,
                a          <- sharedActs,
                q1'        <- getAutNext a1 q1 a,
                q2'        <- getAutNext a2 q2 a
                ]
    prodGraph = Digraph prodNodes prodEdges

    -- Start from any pair of initial states
    startVs = [(q1, q2) | q1 <- initial a1, q2 <- initial a2]

    -- Target: any pair of accepting states
    targets = [(q1, q2) | q1 <- final a1, q2 <- final a2]

-- Check for each A: for all (t1,t2): L(A) \cap L(A_(t1, t2)) = empty?
-- We check the emptiness of the intersection by checking the reachability of the product graph(automaton)
-- If empty, then cond 2 is SAT
checkCond2 :: RegLTSU -> Automaton -> [State] -> [State] -> Bool
checkCond2 m aut phiStates negPsiStates = not (any violates pairs)
  where
    pairs = [(t1, t2) | t1 <- phiStates, t2 <- negPsiStates]
    -- Check the intersection. Non-empty => cond 2 is UNSAT.
    violates (t1, t2) =
        let pathAut = buildPathAutomaton m t1 t2
        in  intersectionNonEmpty aut pathAut
```

```
-- [[phi]]= set of states that phi holds
truthSet :: RegLTSU -> RegForm -> [State]
truthSet m f = [s | s <- statesM m, isTrueReg (m, s) f]

-- Satisfaction relation for the propositional fragment
isTrueReg :: (RegLTSU, State) -> RegForm -> Bool
isTrueReg (m, s) (P p) =
    case lookup s (valuationM m) of
        Just props -> p `elem` props
        Nothing -> False
isTrueReg (m, s) (Neg f) =
    not (isTrueReg (m, s) f)
isTrueReg (m, s) (Disj f g) =
    isTrueReg (m, s) f || isTrueReg (m, s) g

-- Kh_a(phi, psi) holds iff there exists A in U_a such that
-- (1) [[phi]] is subset of SE(L(A))
-- (2) R_{L(A)}([[phi]]) is subset of [[psi]]
isTrueReg (m, _) (KH agent phi psi) =
    any (\aut -> checkCond1 m aut phiStates
              && checkCond2 m aut phiStates negPsiStates
    ) (getAgentAuts m agent)
  where
    phiStates    = truthSet m phi          -- [[phi]]
    negPsiStates = truthSet m (Neg psi)    -- [[neg psi]]

-- Infix alias for the satisfaction relation
(||=) :: (RegLTSU, State) -> RegForm -> Bool
(||=) = isTrueReg
```

5 Tests for Semantics

We now use the library QuickCheck to randomly generate models and states to verify the fundamental logical facts within the boolean fragment of our implementation.

```
module Main where
```

```
import Test.QuickCheck
import Data.List.NonEmpty (toList)
import SingleAgent
import Test.Hspec
```

The following uses the HSpec library to define our test suite. These tests ensure that our model checker respects classical propositional identities. To perform these tests, we first define an auxiliary type `PointedModel` and an `Arbitrary` instance to pick a random state from a generated model.

```
newtype PointedModel = PM (AbilityMap, State) deriving (Show)

instance Arbitrary PointedModel where
    arbitrary = do
        m <- arbitrary
        s <- elements (toList (states m))
        return (PM (m, s))
```

We verify the following principles which should hold for any boolean formula φ:

- **Double Negation Elimination**
  - $\neg\neg\varphi \equiv \varphi$
- **Commutativity of Conjunction**
  - $\varphi \wedge \psi \equiv \psi \wedge \varphi$
- **Identity of Conjunction**
  - $\varphi \wedge \top \equiv \varphi$

```
-- Since Kh is currently undefined, we use a helper function to filter out formulas containing the Kh modality.
isBoolean :: Form -> Bool
isBoolean (P _)        = True
isBoolean T            = True
isBoolean (Neg f)      = isBoolean f
isBoolean (Conj f1 f2) = isBoolean f1 && isBoolean f2
isBoolean (KH _ _)     = False

-- Check if two formulas are equivalent in a specific model and state
isEquivalent :: (AbilityMap, State) -> Form -> Form -> Bool
isEquivalent (m, s) f1 f2 = isTrue (m, s) f1 == isTrue (m, s) f2

main :: IO ()
main = hspec $ do
  describe "Boolean Semantics" $ do
    it "Double Negation Elimination: Neg (Neg f) should be equivalent to f" $
      property $ \(PM (m, s)) f ->
        isBoolean f ==> isEquivalent (m, s) (Neg (Neg f)) f

    it "Commutativity of Conjunction: f1 ^ f2 should be equivalent to f2 ^ f1" $
      property $ \(PM (m, s)) f1 f2 ->
        (isBoolean f1 && isBoolean f2) ==> isEquivalent (m, s) (Conj f1 f2) (Conj f2 f1)

    it "Identity of Conjunction: f ^ T should be equivalent to f" $
      property $ \(PM (m, s)) f ->
        isBoolean f ==> isTrue (m, s) (Conj f T) === isTrue (m, s) f
```

6 Conclusion

Finally,

References

[1] C. Areces, R. Fervari, A. R. Saravia, and F. R. Velázquez-Quesada. Uncertainty-based semantics for multi-agent knowing how logics. In *Proceedings of the 18th Conference on Theoretical Aspects of Rationality and Knowledge (TARK 2021)*, volume 335 of *EPTCS*, pages 23–37, 2021.

[2] S. Demri and R. Fervari. Model-checking for ability-based logics with constrained plans. In *Proceedings of the 37th AAAI Conference on Artificial Intelligence (AAAI-23)*, volume 37, pages 6305–6312, 2023.

[3] Y. Wang. A logic of knowing how. In *Logical Reasoning and Interaction (LORI'15)*, volume 9394 of *Lecture Notes in Computer Science (LNCS)*, pages 392–405. Springer, 2015.